

LangChain 框架常见API总结

一、Model 模型

LangChain支持的模型有三大类

- 1.大语言模型（LLM），也叫非对话模型

也叫文本补全模型，接收文本字符串作为输入，并返回一个补全字符串作为输出。

例如给定一个提示“今天的天气如何?”模型会生成一个相应的答案“今天的天气很好。”

可以理解为只能一问一答。

- 2.聊天模型（Chat Model），也叫对话模型

主要代表Open AI的ChatGPT系列模型。这些模型通常由语言模型支持，但它们的API更加结构化。聊天模型包装器为 ChatOpenAI。

具体来说，ChatOpenAI将一系列的聊天消息/聊天消息列表作为输入，并返回聊天消息。

代码如下：

```
# 封装openai规范的模型对象
# model = ChatOpenAI(api_key=os.getenv("DASHSCOPE_API_KEY"), model="qwen-max", base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")

# 不是兼容openAI的模型，使用init_chat_model创建，兼容任何的大模型
# 如果确定模型是兼容openai的，如果模型切换比较频繁，建议使用init_chat_model创建
# model = init_chat_model(model="deepseek-chat", model_provider="deepseek")

# 确定使用某一个平台的模型
model = ChatTongyi()
# ChatDeepSeek()
```

- 3.文本嵌入模型（Embedding Model）

在Rag部分具体介绍实现

二、提示词模板

1. PromptTemplate：常用的String提示模板
2. 聊天提示模板 ChatPromptTemplate：常用的Chat提示模板，用于组合各种角色的消息模板，传入聊天模型。消息模板包括：ChatMessagePromptTemplate、HumanMessagePromptTemplate、AIMessagePromptTemplate、SystemMessagePromptTemplate等

```
from langchain_community.chat_models import ChatTongyi
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate, AIMessagePromptTemplate

# 模型客户端
model = ChatTongyi()
```

```

# 提示词模板
# prompt = PromptTemplate(
#     template="你是一个翻译助手，请帮我把一下内容翻译成{language}: {text}"
#     , input_variables=["text", "language"]
# )

# 对话模型，角色的设置，多轮对话记忆维持
# 提示词角色: system 全局，统一设置    user 用户每次提问    assistant 大模型回复，平常上
# 下文存储

prompt = ChatPromptTemplate.from_messages([
    # 系统提示词
    SystemMessagePromptTemplate.from_template("你是一个翻译助手，可以翻译任何一种
语言"),
    # 用户提示词
    HumanMessagePromptTemplate.from_template("请将一下内容翻译成{language}:
{text}"),
    # 助手提示词
    AIMessagePromptTemplate.from_template("{text}"),
])

# 给参数赋值
# fact = prompt.format(text="hello world", language="中文")
# fact2 = prompt.format(text="我爱你", language="日文")

# 执行，拿结果
# result = model.invoke(fact)
# result2 = model.invoke(fact2)
# print(result)
# print(result2)

# 结果解析器,格式化输出
out = StrOutputParser()

# print(out.invoke(result))

# 底层还是函数调令，简化书写，固定的流程的调用，格式化
chain = prompt | model | out

print(chain.invoke({"text": "hello world", "language": "中文"}))

```

3. 样本提示模板 FewShotPromptTemplate: 通过少量样本来教模型如何回答

```

#少样本提示模版的使用
from langchain.prompts import PromptTemplate
from langchain.prompts.few_shot import FewShotPromptTemplate
from models import get_lc_model_client

#获得访问大模型客户端
client = get_lc_model_client()

# 创建示例
examples = [
    {"sinput": "2+2", "soutput": "4", "sdescription": "加法运算"},

```

```

    {"sinput": "5-2", "soutput": "3", "sdescription": "减法运算"},
]

# 配置一个提示模板，用来一个示例格式化
examples_prompt_tmplt_txt = "算式: {sinput} 值: {soutput} 类型: {sdescription} "

# 这是一个提示模板的实例，用于设置每个示例的格式
prompt_sample = PromptTemplate.from_template(examples_prompt_tmplt_txt)

# 创建少样本示例的对象
prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=prompt_sample,
    prefix="你是一个数学专家，能够准确说出算式的类型，",
    suffix="现在给你算式: {input}， 值: {output}，告诉我类型: ",
    input_variables=["input", "output"]
)
print(prompt.format(input="2*5", output="10")) # 你是一个数学专家,算式: 2*5 值: 10

print('-' * 50)

result = client.invoke(prompt.format(input="2*5", output="10"))
print(result.content) # 使用: 乘法运算

```

三、输出解析器

输出解析器负责获取 LLM 的输出并将其转换为更合适的格式。

LangChain有许多不同类型的输出解析器：CommaSeparatedListOutputParser、DatetimeOutputParser、JsonOutputParser、XMLOutputParser等等

1.CommaSeparatedListOutputParser：返回列表形式，用逗号分隔返回结果

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import CommaSeparatedListOutputParser,
StrOutputParser
from models import get_lc_model_client

# 获得访问大模型客户端
client = get_lc_model_client()

# 创建解析器
# output_parser = StrOutputParser()
output_parser = CommaSeparatedListOutputParser()

# 提示模板
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个专业的程序员"),
    ("user", "{input}")
])

# 将提示和模型合并以进行调用
chain = prompt | client | output_parser

# 示例调用

```

```
#明确告诉大模型用逗号分隔返回，我们可以用CommaSeparatedListOutputParser获得内容后以
# 列表的形式获得以进行后续处理，否则返回一个字符串
print(chain.invoke({"input": "列出Python的三个主要版本，用逗号分隔"}))
print(chain.invoke({"input": "列举三个常见的机器学习框架，用逗号分隔"}))
```

2.DatetimeOutputParser 按照日期格式返回结果

```
from langchain.output_parsers import DatetimeOutputParser
from langchain.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

from models import get_lc_model_client

#获得访问大模型客户端
client = get_lc_model_client()

# 定义模板格式
template = """
回答用户的问题: {question}

{format_instructions}
"""

# 使用日期时间解析器
output_parser = DatetimeOutputParser()

prompt = PromptTemplate.from_template(
    template,
    # 指定日期的格式类型 yyyy-MM-dd hh:mm:ss
    partial_variables={"format_instructions":
output_parser.get_format_instructions()},
)
print(prompt)
print("-"*30)
print(output_parser.get_format_instructions())
print("-"*30)
# 链式调用
chain = prompt | client | output_parser
output = chain.invoke({"question": "新中国是什么时候成立的? "})
# 打印输出
print(output) # 1949-10-01

# 执行
# output = model.invoke(prompt.format(question='新中国成立时间? '))
# datetime_parsed = output_parser.parse(output.content)
# # 打印输出
# print(datetime_parsed) # 1949-10-01
```

3.JsonOutputParser 表示将结果以json格式返回

```
from langchain_core.prompts import ChatPromptTemplate
# 创建解析器
from langchain_core.output_parsers import JsonOutputParser, StrOutputParser
```

```

from models import get_lc_model_client

#获得访问大模型客户端
client = get_lc_model_client()

# output_parser = StrOutputParser()
output_parser = JsonOutputParser()

# 提示模板
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个专业的程序员"),
    ("user", "{input}")
])

# 将提示和模型合并以进行调用
chain = prompt | client | output_parser

#明确告诉大模型用JSON格式返回，我们可以用JSONOutputParser获得JSON格式的内容以进行后续处理，
否则返回一个字符串
result = chain.invoke({"input": "langchain是什么？问题用question 回答用ans 返回一个JSON格式"})
print(type(result))
print(result)
# print(chain.invoke({"input": "大模型中的langchain是什么?"}))

```

四、Langchain的链和LCEL

实际上，LangChain的名字源自其框架的核心设计思路：

用链(Chain)，将大语言模型开发的各个组件链接起来，以构建复杂的应用程序。

链的主要功能是管理应用程序中的数据流动，它将不同的组件(或其他链)链接在一起，形成一个完整的数据处理流程。每个组件都是链中的一个环节，它们按照预设的顺序，接力完成各自的任务，就好比工厂里的流水线。

LangChain早期的链只有一种实现方式，但是后来LangChain引入了LCEL链逐渐代替原有的实现。

所以在V0.1以后，开始对链进行了分类，分为了遗留链和LCEL链。在LangChain官方将所有的旧链全部替换为LCEL链之前，遗留链暂时可用。

LCEL 也被称为LangChain表达式(LangChain Expression Language)，是一种用声明式的方法来链接LangChain组件。

所有可以被链起来的组件，如大语言模型、输出解析器、检索器、提示词模板等都支持下面三个方法：

stream: 流式返回响应的块

invoke: 接受输入返回输出 invoke 调用

batch: 接受批量输入返回输出列表

所以最终组成的链在使用上，也是调用这三个方法。

1.链的基本使用

如何把组件链起来？

最简单最常见的就是用管道操作符“|”，在前面的代码中，我们已经多次使用过了

```
#获得访问大模型客户端
client = get_lc_model_client()

# output_parser = StrOutputParser()
output_parser = JsonOutputParser()

# 提示模板
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个专业的程序员"),
    ("user", "{input}")
])

# 通过管道符 | 实现链的组合
chain = prompt | client | output_parser

#明确告诉大模型用JSON格式返回，我们可以用JSONOutputParser获得JSON格式的内容以进行后续处理，
否则返回一个字符串
result = chain.invoke({"input": "langchain是什么？ 问题用question 回答用ans 返回一个JSON格式"})
```

2.链的常用组件：

RunnableSequence 顺序执行链中的每个节点，类似|管道符

```
import os

# debug模式的包
import langchain
from langchain.chat_models import init_chat_model
from langchain_community.chat_models import ChatTongyi, ChatHunyuan
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate,
    SystemMessagePromptTemplate,
    HumanMessagePromptTemplate, AIMessagePromptTemplate
from langchain_core.runnables import RunnableSequence
from langchain_openai import ChatOpenAI

# 链式调用
# 1.创建模型对象，基于OpenAI 规范
# 作用：跟踪链路，跟踪每个节点的输入，输出 调试程序时使用
# langchain.debug=True

model = ChatTongyi()

# 2.构建提示词
```

```

# 提示词模板 ctrl+鼠标左键
prompt = PromptTemplate(
    template="你是一个翻译助手，请帮我把一下内容翻译成{language}: {text}"
    , input_variables=["text", "language"]
)

# 格式化返回结果,字符串格式化,content内容提取
out = StrOutputParser()

# 构建链 |管道 linux命令中有管道,langchain框架预制了很多工作链
# chain = prompt | model | out
chain = RunnableSequence(prompt, model, out)

# 链的调用
result = chain.invoke({"language": "英文", "text": "我喜欢编程"})

print(result)

```

RunnableLambda

很多的时候LangChain组件未提供的功能，需要我们自己写函数实现，但是这个函数我们也需要把它加入链，作为工作流程的一部分，此时就需要使用RunnableLambda,装饰器@chain是RunnableLambda的另一种写法

```

# RunnableLambda的使用
from operator import itemgetter
import langchain
from langchain_community.chat_models import ChatOllama, ChatHuggingFace
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda
from langchain_core.runnables import chain
from models import get_lc_model_client

#开启该参数，会输出调试信息
# langchain.debug = True
#获得访问大模型客户端
client = get_lc_model_client()

# langchain调用本地部署的模型
# client = ChatOllama()
# ChatHuggingFace()

#定义提示模版
chat_template = ChatPromptTemplate.from_template(" {a} + {b}是多少? ")

output = StrOutputParser()
#获得字符串的长度 没有继承Runnable类
def length_function(text):
    return len(text)

#将两个字符串长度的数量相乘

```

```

def _multiple_length_function(text1, text2):
    return len(text1) * len(text2)

#@chain是RunnableLambda的另一种写法
@chain
def multiple_length_function(_dict):
    return _multiple_length_function(_dict["text1"], _dict["text2"])

# chain1 = chat_template | client | output
chain1 = chat_template | client

#使用RunnableLambda将函数转换为与LCEL兼容的组件

# RunnableLambda 调用自定义函数，自定义函数的功能可以其他工具进行参数的获取
chain2 = (
    # a=3 b=12
    {
        "a": itemgetter("foo") | RunnableLambda(length_function),
        # "a": itemgetter("foo") | length_function,
        "b": {"text1": itemgetter("foo"), "text2": itemgetter("bar")}} |
    multiple_length_function,
    }
    | chain1
)
print(chain2.invoke({"foo": "abc", "bar": "abcd"}))

#模拟用户的业务，可以从数据库、其他文件中获得数据

# itemgetter 根据key获取字典中的值    a=3 b=12
chain3 = (
    {
        "a": ( itemgetter("foo") | RunnableLambda(length_function) ),
        "b": ( {"text1": itemgetter("foo"), "text2": itemgetter("bar")} |
    multiple_length_function )
    }
    | chain1 ) | output
print(chain3.invoke({"foo": "abc", "bar": "abcd"}))

```

RunnableParallel

把组件链起来，前面我们使用了RunnableSequence。Sequence的中文意思“顺序，一连串”，也就是所有的组件或多个任务都是按顺序一个个执行的。

但是有时候，我们希望多个任务能够同时执行，这时可以使用RunnableParallel

```

#RunnableParallel的使用
from langchain_core.runnables import RunnableLambda, RunnableParallel,
RunnableMap, RunnableSequence

def add_one(x: int) -> int:
    return x + 1

def mul_two(x: int) -> int:
    return x * 2

```



```

def mul_three(x: int) -> int:
    return x * 3

# 测试RunnableParallel, RunnableMap 并行执行
# chain = RunnableSequence(add_one,mul_two,mul_three)    12 原因: RunnableSequence
封装成了Runnable实现
# print(chain.invoke(1))

# 表示 add_one 执行的同时 mul_two, mul_three 两个都会执行 返回值是一个字典, 通过key
chain = RunnableParallel(
    a =add_one,
    b = mul_two,
    c = mul_three
)

# chain = RunnableMap(
#     a = add_one,
#     b = mul_two,
#     c = mul_three
# )
# # 字典结构 {'a': 3, 'b': 4, 'c': 6}    java 中map
# print(chain.invoke(2))

chain1 = RunnableLambda(add_one)

chain2 = chain1|RunnableParallel(
    a = mul_two,
    b = mul_three
)
# 结果字典: {'a': 6, 'b': 9}
print(chain2.invoke(2))

```

RunnablePassthrough

RunnablePassthrough类允许我们在LangChain的链中传递数据:

- 1、确保数据保持不变, 直接用于后续步骤的输入。RunnablePassthrough常用在链的第一个位置, 用于接收用户的输入 (也可以用在中间位置, 则用于接收上一步的输出)。
- 2、RunnablePassthrough也允许通过assign对数据增强后, 再往后传。

通过 `RunnablePassthrough`, 你可以在 LangChain 中以最小的成本实现数据增强, 让流程保持简洁的同时具备灵活的扩展能力, 尤其适合需要轻量级数据处理的场景 (如日志、元数据添加、临时参数注入)

```

#RunnablePassthrough的使用
#RunnablePassthrough的两种用法都将在我们后面的课程中看到
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

# RunnablePassthrough原样进行数据传递
runnable = RunnableParallel(
    passed=RunnablePassthrough(),
    modified=lambda x: x["num"] + 1,
)

```

```
# #
# print(runnable.invoke({"num": 1}))

# RunnablePassthrough对数据增强后传递
#RunnablePassthrough().assign它会创建一个新的字典，包含原始的所有字段以及你新指定的字段。
runnable = RunnableParallel(
    passed=RunnablePassthrough().assign(query=lambda x: x["num"] + 2),
    modified=lambda x: x["num"] + 1,
)
# {'passed': {'num': 1, 'query': 3}, 'modified': 2}
print(runnable.invoke({"num": 1}))
```

五、记忆 (Memory)

大模型是无状态的，也就是大模型是记不住我们聊天中每次的对话内容的。

验证大模型的无状态：

```
#展示大模型的无状态，记不住我们聊天中每次的对话内容
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate,
SystemMessagePromptTemplate

from models import get_lc_model_client
# 直接访问LLM
client = get_lc_model_client()

chat_template = ChatPromptTemplate.from_messages(
    [
        SystemMessagePromptTemplate.from_template("你是人工智能助手"),
        ('human', '{text}')
    ]
)
parser = StrOutputParser()

chain = chat_template | client | parser

while True:
    user_input = input("请输入 'quit' 退出程序: ")
    if user_input == 'quit':
        print("程序结束。")
        break
    else:
        print(chain.invoke({'text': user_input}))

# # 问题1
# print(chain.invoke({'text': '你好，我是大白'}))
# # 问题2
# print(chain.invoke({'text': '你好，我是谁？'}))
```

ChatMessageHistory的使用

用来在内存中存储对话的上下文，也就是维持多轮对话的“记忆”

ChatMessageHistory比较早期的命名，功能实现是InMemoryChatMessageHistory

常用的API有四个：

add_user_message 存用户消息

add_ai_message 存AI消息

history.messages 获取所有消息

clear 清空所有消息

#消息历史组件ChatMessageHistory的使用

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate,
SystemMessagePromptTemplate, MessagesPlaceholder,
HumanMessagePromptTemplate
from langchain_openai import ChatOpenAI
from langchain_community.chat_message_histories import ChatMessageHistory
from models import get_lc_model_client, get_alibaba_model_client

chat_template = ChatPromptTemplate.from_messages(
    [
        SystemMessagePromptTemplate.from_template("你是人工智能助手"),
        HumanMessagePromptTemplate.from_template("{input}"),
        # 作用就是向提示词中插入一段上下文消息
        # ("placeholder", "{messages}"),
        MessagesPlaceholder(variable_name="messages"),
    ]
)

client = get_alibaba_model_client()

parser = StrOutputParser()
chain = chat_template | client | parser

# 创建消息历史记录
chat_history = ChatMessageHistory()

while True:
    user_input = input("用户: ")
    if user_input == "exit":
        break
    # 添加用户输入
    chat_history.add_user_message(user_input)
    # 访问LLM时，chat_history.messages 获取所有的历史消息
    response = chain.invoke({'messages': chat_history.messages, 'input':
user_input})
```

```

print("chat_history:", chat_history.messages)
print(f"大模型回复》》》: {response}")
# 将大模型的回复加入历史记录
chat_history.add_ai_message(response)

# #第一轮对话： 添加用户的提问消息
# chat_history.add_user_message('你好，我是大白')#用户提问加入历史记录
# response = chain.invoke({'messages': chat_history.messages})
#
# print(response)
# #第一轮对话： 添加模型应答消息
# chat_history.add_ai_message(response) #模型应答加入历史记录
# print("chat_history:", chat_history.messages)
#
# # 第二轮对话： 添加用户的提问消息
# chat_history.add_user_message('你好，我是谁? ')#用户提问加入历史记录
# print(chain.invoke({'messages': chat_history.messages}))

```

RedisChatMessageHistory

将记忆存储在内存，系统重启或者电脑关机将清空内存，这个时候记忆也就自动清除了，能不能实现长期保存？

答案是肯定的，可以使用关系型数据库、ES、消息队列等保存，也可以使用内存数据库保存，redis数据库是比较常用的一种。

```

# 1.pip install -qu langchain-redis langchain-openai redis
from gradio.themes.builder_app import history
from langchain_redis import RedisChatMessageHistory

from models import get_lc_model_client
# session_id 识别用户 redis_url 访问路径
history = RedisChatMessageHistory(session_id="my_session_id",
redis_url="redis://localhost:6379")

# history.clear() 清空历史消息

client = get_lc_model_client()
# history.add_user_message("你是谁? ")
#
# amessage = client.invoke(history.messages)
# history.add_ai_message(ameessage)
# print(ameessage)

history.add_user_message("重复一次")
print(history.messages)
ameessage = client.invoke(history.messages)
history.add_ai_message(ameessage)
print(ameessage)

# RedisChatMessageHistory 和 ChatMessageHistory 都有相同的API

```

RunnableWithMessageHistory

`RunnableWithMessageHistory` 是 **LangChain/LangGraph** 框架中的一个核心组件，专门用于为链式操作 (Chain) 或可运行对象 (Runnable) **添加对话历史管理能力**。它使得构建能够记住多轮对话上下文的聊天机器人或对话系统变得非常简单。它能在多次调用之间**自动维护和注入对话历史记录**，让LLM能够基于完整的对话上下文生成回复。

```
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory
```

```
# 首先创建一个基础的Runnable (比如一个简单的链)
client = get_lc_model_client()

prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个有用的助手。"),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])
chain = prompt | client
```

#2. 创建RunnableWithMessageHistory

```
chain_with_history = RunnableWithMessageHistory(
    chain, # 基础的链
    get_session_history, # 获取历史记录的函数
    input_messages_key="input", # 输入消息的键
    history_messages_key="history" # 历史消息的键
)
```

#3. 定义获取历史记录的函数

```
store = {} # 简单的内存存储

def get_session_history(session_id: str) -> ChatMessageHistory:
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]
```

#4. 使用 (必须指定session_id来区分不同对话)

```
response = chain_with_history.invoke(
    {"input": "你好, 我叫小明"},
    config={"configurable": {"session_id": "user123"}}
)
```

六、RAG实现

在许多LLM应用程序中，用户特定的数据不在大模型中，可能在外部系统或文档中。如何使用这些外部数据来增强呢？

LangChain中的数据连接组件包括几个关键模块：文档加载器、文档切分、文本嵌入、向量存储、检索器 等等。

1.文档与文档加载与切割

文档类型有很多，可以是简单的文本文件、Word文档、Excel表格、Pdf文件等等，甚至是 各种视频的转录文件

加载器有 load方法，用于从指定的数据源读取数据，并将其转换成一个或多个文档。这使得 LangChain 能够处理各种形式的输入数据，不仅仅限于文本数据，还可以是网页、视频字幕等。

LangChain有很强的数据加载能力，提供了很多常见的数据格式的支持，例如CSV、文件目录、HTML、JSON、Markdown及PDF等。

TXT文档:TextLoader

PDF文档:PyPDFLoader

CSV文档:CSVLoader

JSON文档:JSONLoader

HTML文档:UnstructuredHTMLLoader

MD文档:UnstructuredMarkdownLoader

文件目录:DirectoryLoader

在前面的RAG基础章节，我们已经了解了文本的分割。LangChain为此也提供了很多现成的分割器：

CharacterTextSplitter： 基于字符(默认为"\n\n")进行切割，并通过字符数量来测量文本块的大小。使用chunk_size属性可设置文本块的大小，使用chunk_overlap 属性设置文本块之间的最大重叠。

RecursiveCharacterTextSplitter： 在前面我们已经了解过，除了自然语言，它也支持对特定的编程语言('JavaScript'、'cpp'、'go'、'java'、'php'、'python'等等)的代码进行切割。

MarkdownHeaderTextSplitter： 可以根据指定的一组标题来切割一个Markdown 文档。

.....

内置分割器： https://python.langchain.com/api_reference/text_splitters/index.html

```
# 1.指定要加载的word文档路径
loader = Docx2txtLoader("人事管理流程.docx")

# 加载文档、转换格式化成document
documents = loader.load()
# print(len(documents))
# # Document(metadata={'source': '人事管理流程.docx'}, page_content='集团管理制度\n\n人')
# print(documents)

# 文档切割 递归切割
# separators
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,

    # separators=[分隔符]
)
# 通过分割器获取document :create_documents    split_documents    传入一个document对象，返回一个document对象列表
split_documents = text_splitter.split_documents(documents)
```

2.文本嵌入模型（Embedding Model）

LangChain 框架提供了一个名为Embeddings 的类，它为多种文本嵌入模型(如OpenAI、Cohere、HuggingFace 等)提供了统一的接口。通过该类实例化的嵌入模型包装器，可以将文档转换为向量数据，同时将搜索的问题也转换为向量数据，这使得可通过计算搜索问题和文档在向量空间中的距离，来寻找在向量空间中最相似的文本。

```
# 模型包装器：大模型分成三类：LLM 聊天模型 嵌入模型
# 获得一个阿里通义千问嵌入模型的实例，同样在models.py中被包装为get_alibaba_embeddings()
from langchain_community.embeddings import DashScopeEmbeddings
# 嵌入模型的模型包装器
llm_embeddings = DashScopeEmbeddings(
    # 模型名称
    model=ALI_TONGYI_EMBEDDING_MODEL,
    # API_KEY
    dashscope_api_key=os.getenv(ALI_TONGYI_API_KEY_OS_VAR_NAME)
)
```

3.向量数据库：

```
# 实例化向量空间，向量化+向量存储到向量数据库中
vector_store =
Chroma.from_documents(documents=split_documents, embedding=llm_embeddings)

#展示相似度查询，实际业务中可以不要
# print(vector_store.similarity_search("狸花猫"))
# print("--"*15)
# #按相似度的分数进行排序，分数值越小，越相似（其实是L2距离）
# print(vector_store.similarity_search_with_score("狸花猫"))
```

4.检索器

检索器 (Retriever) 是 RAG (检索增强生成) 系统的核心组件，负责从知识库中查找与用户查询相关的文档片段。

1.创建 FAISS 检索器

```
retriever_faiss = FAISS.from_texts(texts, embeddings).as_retriever()
```

2.创建 Chroma 检索器

```
retriever_chroma = Chroma.from_documents(docs, embeddings).as_retriever()
```

3.关键词检索器

```
retriever_bm25 = BM25Retriever.from_texts(texts)
```

4.混合检索器 (结合语义+关键词)

```
ensemble_retriever = EnsembleRetriever(
    retrievers=[retriever_faiss, retriever_bm25],
    weights=[0.7, 0.3] # 权重分配
)
```

检索器底层默认使用的是语义相似性进行检索，可以通过allowed_search_types 参数设置检索方式

VectorStoreRetriever 中**allowed_search_types** 参数设置检索方式：
similarity **similarity_score_threshold** **mmr** 最大边际相似性检索：
mmr 最大边际相似性检索：1.相似性检索；2.根据相似性得分进行过滤+与已选择结果进行相似度匹配
（去除冗余，鼓励多样性）

```
# 实例化向量空间，向量化+向量存储到向量数据库中
vector_store =
Chroma.from_documents(documents=split_documents, embedding=llm_embeddings)

#展示相似度查询，实际业务中可以不要
# print(vector_store.similarity_search("狸花猫"))
# print("--"*15)
# #按相似度的分数进行排序，分数值越小，越相似（其实是L2距离）
# print(vector_store.similarity_search_with_score("狸花猫"))

# 检索器对象：检索文档，可以根据需要对检索后的结果做各种处理
# VectorStoreRetriever 中allowed_search_types 参数设置检索方式：
# similarity similarity_score_threshold mmr 最大边际相似性检索：
# mmr 最大边际相似性检索：1.相似性检索；2.根据相似性得分进行过滤+与已选择结果进行相似度匹配
（去除冗余，鼓励多样性）
retriever = vector_store.as_retriever()
# result = retriever.invoke("晋升")
```

完整的Rag案例：

```
import os
# 安装 pip install langchain_chroma
# 加载word文档 安装 pip install docx2txt
# 加载json文档 安装 pip install jq
# 加载pdf文档 安装 pip install pymupdf
# 加载HTML文档 安装 pip install unstructured
# 加载MD文档 安装 pip install markdown + pip install unstructured
import langchain
# pip install langchain-chroma
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
from langchain_core.documents import Document
from langchain_core.vectorstores import VectorStore
from langchain_text_splitters import RecursiveCharacterTextSplitter

from models import get_lc_model_client, ALI_TONGYI_API_KEY_OS_VAR_NAME,
ALI_TONGYI_EMBEDDING_MODEL, get_ali_model_client

#获得访问大模型客户端
client = get_ali_model_client()
```


#直接了解LangChain中的“文档”(Document)的具体内容，这里我们跳过了文档与文档加载，文档切割和文档转换过程

#文档的模拟数据

```
# documents = [  
#     Document(  
#         page_content="猫是柔软可爱的动物，但相对独立",  
#         metadata={"source": "常见动物宠物文档"},  
#     ),  
#     Document(  
#         page_content="狗是人类很早开始的动物伴侣，具有团队能力",  
#         metadata={"source": "常见动物宠物文档"},  
#     ),  
#     Document(  
#         page_content="金鱼是我们常常喂养的观赏动物之一，活泼灵动",  
#         metadata={"source": "鱼类宠物文档"},  
#     ),  
#     Document(  
#         page_content="鹦鹉是猛禽，但能够模仿人类的语言",  
#         metadata={"source": "飞禽宠物文档"},  
#     ),  
#     Document(  
#         page_content="兔子是小朋友比较喜欢的宠物，但是比较难喂养",  
#         metadata={"source": "常见动物宠物文档"},  
#     ),  
# ]
```

```
from langchain_community.document_loaders import UnstructuredWordDocumentLoader,  
Docx2txtLoader
```

1. 指定要加载的word文档路径

```
loader = Docx2txtLoader("人事管理流程.docx")
```

加载文档、转换格式化成document

```
documents = loader.load()
```

```
# print(len(documents))
```

```
# # Document(metadata={'source': '人事管理流程.docx'}, page_content='集团管理制度\n\n人')
```

```
# print(documents)
```

文档切割 递归切割 separators

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=50,  
)
```

通过分割器获取document :create_documents split_documents 传入一个document对象，返回一个document对象列表

```
split_documents = text_splitter.split_documents(documents)
```

模型包装器：大模型分成三类：LLM 聊天模型 嵌入模型

获得一个阿里通义千问嵌入模型的实例，同样在models.py中被包装为get_ali_embeddings()

```
from langchain_community.embeddings import DashScopeEmbeddings
```

嵌入模型的模型包装器

```
llm_embeddings = DashScopeEmbeddings(  
    # 模型名称  
    model=ALI_TONGYI_EMBEDDING_MODEL,
```

```

# API_KEY
dashscope_api_key=os.getenv(ALI_TONGYI_API_KEY_OS_VAR_NAME)
)

# 实例化向量空间，向量化+向量存储到向量数据库中
vector_store =
Chroma.from_documents(documents=split_documents,embedding=llm_embeddings)

retriever = vector_store.as_retriever()

message = """
仅使用提供的上下文回答下面的问题：
{question}
上下文：
{context}
"""

prompt_template = ChatPromptTemplate.from_messages([('human',message)])
# 用RunnablePassthrough允许我们将用户的具体问题在实际使用过程中进行动态传入
chain = {"question":RunnablePassthrough(),"context":retriever} | prompt_template
| client

#用大模型生成答案
resp = chain.invoke("晋升")
print(resp.content)

```

七、LangChain工具的调用

如果大模型只是和我们进行自然语言聊天，它固然有用，但是决定没有现在这样用途广泛。我们希望大模型能做更多的事，比如像人类一样使用工具

现在比较火爆的MCP、Agent2Agent说到底，都离不开大模型对工具的调用，那么大模型是如何进行工具调用的呢？首先需要让大模型知道有哪些可用的工具，然后才能进行分析判断什么需求可以调用哪些工具，最后才是真正的调用工具，将工具的返回给到大模型，从而生成结果返回。

1.工具Tools的定义

工具是代理、链或LLM可以用来与世界互动的接口。它们结合了几个要素

- 工具的名称
- 工具的描述
- 该工具输入的JSON模式
- 要调用的函数
- 是否应将工具结果直接返回给用户

LangChain通过提供统一框架集成功能的具体实现，在框架内，每个功能被封装成一个工具。Tools 组件中可以调用的各种工具类。LangChain 提供了一组预定义的Tools 组件，同时也允许用户自定义Tools 组件以满足特定的需求。

Toolkits 组件是一个特殊的组件集合，它包括了多个用于实现特定目标的 Tools组件，LangChain 也提供了预定义的 Toolkits 组件。

工具定义：

```

@tool
def get_date():

```

```

        """ 获取北京今天的天气 """
        return datetime.date.today().strftime("%Y-%m-%d")

# schema 英语，语法：作用定义英语语句改怎么说怎么写    xml

import webbrowser
@tool
def open_browser(url, browser_name=None):
    """ 获取浏览器，打开网站 """
    if browser_name:
        # 获取特定浏览器的控制器
        browser = webbrowser.get(browser_name)
    else:
        # 使用默认浏览器
        browser = webbrowser
    # 打开浏览器并导航到指定的URL
    browser.open(url)

```

大模型绑定工具：

```

# api_key 密钥  model: 模型名称  base_url: 模型连接的url地址
model = ChatOpenAI(api_key=os.getenv("DASHSCOPE_API_KEY"), model="qwen-max",
base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")

# resp = model.invoke("今天是几月几号? ")
# print(resp)

@tool
def get_date():
    """ 获取北京今天的天气 """
    return datetime.date.today().strftime("%Y-%m-%d")

# schema 英语，语法：作用定义英语语句改怎么说怎么写    xml

import webbrowser
@tool
def open_browser(url, browser_name=None):
    """ 获取浏览器，打开网站 """
    if browser_name:
        # 获取特定浏览器的控制器
        browser = webbrowser.get(browser_name)
    else:
        # 使用默认浏览器
        browser = webbrowser
    # 打开浏览器并导航到指定的URL
    browser.open(url)

# 大模型客户端绑定工具
tool_llm = model.bind_tools([get_date, open_browser])

```

2.Langchain工具的调用

通过Langchain1.0中的create_agent来实现工具调用，其中参数tools就是用来绑定工具列表

```
import datetime
import os

from dashscope import api_key
from langchain.agents import create_agent

from langchain_openai import ChatOpenAI
from langchain.chat_models import init_chat_model
from langchain_core.prompts import PromptTemplate
# from langchain_core.tools import tool
from langchain.tools import tool

# api_key 密钥 model: 模型名称 base_url: 模型连接的url地址
model = ChatOpenAI(api_key=os.getenv("DASHSCOPE_API_KEY"), model="qwen-max",
base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")

# 注意：函数的描述必须写在函数体中的第一行
@tool
def get_date():
    """ 获取今天的具体日期 """
    # """ 获取今天的北京的天气 """
    return datetime.date.today().strftime("%Y-%m-%d")

import webbrowser
@tool
def open_browser(url, browser_name=None):
    """ 获取浏览器，打开网站 """
    if browser_name:
        # 获取特定浏览器的控制器
        browser = webbrowser.get(browser_name)
    else:
        # 使用默认浏览器
        browser = webbrowser
    # 打开浏览器并导航到指定的URL
    browser.open(url)

# 大模型客户端绑定工具
agent = create_agent(
    model,
    tools=[get_date, open_browser],
)
# 执行agent
# result = agent.invoke({"messages":[{"role":"user","content":"帮我打开淘宝"}]})
# result = agent.invoke({"messages":[{"role":"user","content":"今天是几月几号? "}]})
# 获取北京的今天的天气
result = agent.invoke({"messages":[{"role":"user","content":"帮我打开淘宝"}]})
print( result)
```

3.Langchain中预定义工具的使用

Langchain中定义了很多工具，我们现在以arxiv工具为例给大家演示，arxiv工具是用来在arxiv.org网站查询检索论文的，预定义的工具Langchain已经定义好了，我们只需要使用load_tools()方法导入arxiv工具就行，工具具体的实现类是 ArxivQueryRun(BaseTool)

```
import os
import re

# 验证模型连接
from langchain_community.chat_models import ChatTongyi
from langchain_core.messages import HumanMessage

llm = ChatTongyi(api_key=os.environ.get("DASHSCOPE_API_KEY"))

# 构建一个基于arxiv工具的论文查询智能体，实现根据论文编号查询论文信息的功能。
# 使用load_tools()方法导入arxiv工具: class ArxivQueryRun(BaseTool)
from langchain_community.agent_toolkits.load_tools import load_tools

# 导入arxiv工具
tools = load_tools(["arxiv"])

# 短期记忆构建：使用InMemorySaver()实现单会话的短期记忆
from langgraph.checkpoint.memory import InMemorySaver
# 创建短期记忆实例
memory = InMemorySaver()
# 系统提示词设计：简洁明了地定义Agent的角色和行为准则
system_prompt = "你是一个专业的论文查询助手，使用arxiv工具为用户查询论文信息，回答需简洁准确，包含论文标题、作者、发表时间和核心摘要。"
# 组装并调用Agent
# 使用create_agent()方法组装Agent，并通过invoke()方法调用。
from langchain.agents import create_agent

# 组装Agent
agent = create_agent(
    model=llm,
    tools=tools, #添加工具列表，绑定的论文查询的工具
    system_prompt=system_prompt,
    checkpointer=memory # 传入记忆组件
)

# 调用Agent查询论文
result = agent.invoke(
    {"messages": [{"role": "user", "content": "请查询arxiv论文编号1605.08386的信息"}]},
    # 配置会话标识，用于区分不同用户
    config={"configurable": {"thread_id": "user_1"}} # 会话唯一标识，用于区分不同用户
)

# 输出结果（取最后一条消息的内容）
print(result["messages"][-1].content)
"""
```

执行上述代码后，Agent会自动完成以下流程：

```
接收用户查询，识别需要调用arxiv工具
生成工具调用请求，传入论文编号1605.08386
执行arxiv工具，获取论文信息（标题、作者、发表时间、摘要）
整合工具返回结果，生成自然语言回复
"""
```

4.其他工具调用方式

langchain中可以通过内置链的形式调用工具，以下是调用工具查询数据库，通过自然语言查询mysql数据库：

```
import os
from operator import itemgetter

import langchain
from langchain.chains.sql_database.query import create_sql_query_chain
from langchain_community.tools import QuerySQLDatabaseTool
from langchain_community.utilities import SQLDatabase

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_openai import ChatOpenAI
import re
from models import get_lc_model_client, get_alibaba_model_client,
ALI_TONGYI_MAX_MODEL

#获得访问大模型客户端
client = get_alibaba_model_client(model=ALI_TONGYI_MAX_MODEL)
#数据库配置
HOSTNAME = '127.0.0.1'
PORT = '3306'
DATABASE = 'world'
USERNAME = 'root'
PASSWORD = '1234'
MYSQL_URI = 'mysql+mysqldb://{user}:{password}@{host}:{port}/{db}?
charset=utf8mb4'.format(USERNAME, PASSWORD, HOSTNAME, PORT, DATABASE)
db = SQLDatabase.from_uri(MYSQL_URI)
```

'''对于问题： "请从国家表中查询出China的所有数据"，要分为几步才能出结果：

- 1、大模型判断这个问题需要调用工具查询数据库，获得所有的表名和表中的字段名，目的是看那个表才是国家表，国家表有哪些字段，
- 2、工具执行后，把工具执行结果交给大模型
- 3、大模型根据国家表和国家表中的字段，生成SQL语句
- 4、SQL语句的执行依然需要使用工具
- 5、工具执行后，把工具执行结果交给大模型，大模型生成最终答案'''

#2、因为实际产生的sql是形如``sql....``的，无法直接执行，所以需要清理
#自定义一个输出解析器SQLCleaner

```

class SQLCleaner(StrOutputParser):
    def parse(self, text: str) -> str:
        pattern = r'```sql(.*)```'
        match = re.search(pattern, text, re.DOTALL)
        if match:
            sql = match.group(1).strip()
            # 某些大模型还会产生类似'SQLQuery:'前缀, 必须去除
            sql = re.sub(r'^SQLQuery:', '', sql).strip()
            return sql
        # 某些大模型还会产生类似'SQLQuery:'前缀, 必须去除
        text = re.sub(r'^SQLQuery:', '', text).strip()
        return text

```

```

#1、用LangChain内置链create_sql_query_chain将大模型和数据库结合, 会产生sql而不会执行sql
# 通过create_sql_query_chain将步骤中的1、2、3合起来一起做了
# sql_make_chain = create_sql_query_chain(client, db)
# resp = sql_make_chain.invoke({"question": "请从国家表中查询出China的所有数据"})
# print("产生的SQL语句: ", resp)
# print("*****15)
# 预定义的链, 通用的功能, langchain已经将实现流程固定, 代码已经定义好
sql_make_chain = create_sql_query_chain(client, db) | SQLCleaner()

```

```

resp = sql_make_chain.invoke({"question": "请从国家表中查询出China的相关数据"})
print("实际可用SQL: ", resp)
# print("*****15)

```

#3、将前面的部分组合起来, 得到最终结果

```

answer_prompt = PromptTemplate.from_template(
    """给定以下用户问题、可能的SQL语句和SQL执行后的结果, 回答用户问题
    Question: {question}
    SQL Query: {query}
    SQL Result:{result}
    回答: """
)

# 创建一个执行SQL的工具
execute_sql_tools = QuerySQLDatabaseTool(db = db)
# runnable = RunnablePassthrough.assign(query=sql_make_chain)
# print("RunnablePassthrough-1: ", runnable.invoke({"question": "请从国家表中查询出China的相关数据"}))
#
# runnable = RunnablePassthrough.assign(query=sql_make_chain) |
# itemgetter('query')
# print("RunnablePassthrough-2: ", runnable.invoke({"question": "请从国家表中查询出China的相关数据"}))
#
# runnable = RunnablePassthrough.assign(query=sql_make_chain) |
# itemgetter('query') | execute_sql_tools
# print("RunnablePassthrough-3: ", runnable.invoke({"question": "请从国家表中查询出China的相关数据"}))
#
# runnable =
RunnablePassthrough.assign(query=sql_make_chain).assign(result=itemgetter('query') | execute_sql_tools)

```

```

# print("RunnablePassthrough-4: ",runnable.invoke({"question":"请从国家表中查询出
China的相关数据"}))
#
# exit()
'''通过上面的步骤，就能搞清楚{question}、{query}、{result}这三个字段是如何通过LCEL链一步步
获得的
要注意的是result=itemgetter('query')|execute_sql_tools 中，执行顺序是：
itemgetter('query') -> execute_sql_tools -> result=
所以这段代码实际是：result=(itemgetter('query')|execute_sql_tools)'''
chain =
(RunnablePassthrough.assign(query=sql_make_chain).assign(result=itemgetter('quer
y')|execute_sql_tools)
    |answer_prompt| client| StrOutputParser())

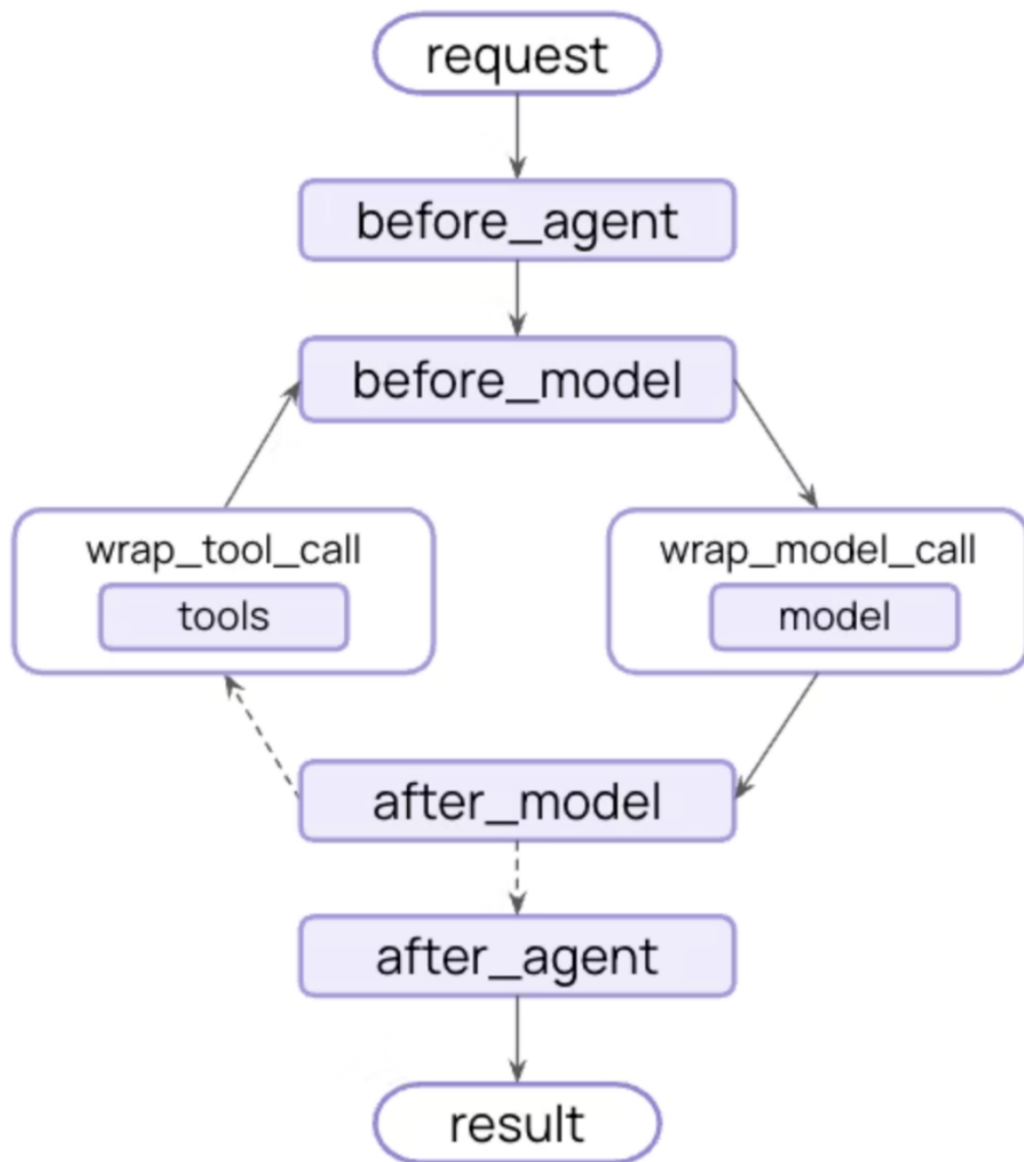
result = chain.invoke(input={"question":"请从国家表中查询出China的相关数据"})
#result = chain.invoke(input={"question":"请问国家表中有多少条数据"})
print("最终执行的结果: ",result)
'''，如果场景是确定的，并不需要大模型来决定是否使用工具，直接在链中加入工具即可
#但是如果需要大模型来决定是否使用工具，比如场景是动态的或者是以工具组的形式提供工具，那么需要使用
Function Call: '''

```

八、中间件

1.什么是中间件Middleware?

是一种细粒度流程控制机制，用于在智能体执行过程中拦截、修改或增强请求与响应的处理逻辑，而无需修改核心 Agent 或工具的代码。可以在调用模型或者工具或者Agent的前后进行拦截，也可以根据需求只做前置拦截或者后置拦截



2.内置中间件

Langchain中内置的中间件很多，以下列出了一部分中间件的介绍：

Middleware 中间件	Description 描述
<u>Summarization 总结</u>	Automatically summarize conversation history when approaching token limits. 当接近会话次数上限时，自动汇总对话历史记录。
<u>Human-in-the-loop 人机交互</u>	Pause execution for human approval of tool calls. 暂停执行，等待人工审核工具调用。
<u>Model call limit 模型调用限制</u>	Limit the number of model calls to prevent excessive costs. 限制模型调用次数，以防止成本过高。
<u>Tool call limit 工具调用限制</u>	Control tool execution by limiting call counts. 通过限制调用次数来控制工具执行。
<u>Model fallback 模型回退</u>	Automatically fallback to alternative models when primary fails. 当主模型发生故障时，自动回退到备用模型。
<u>PII detection PII 检测</u>	Detect and handle Personally Identifiable Information (PII). 检测和处理个人身份信息（PII）。

以下我们就以SummarizationMiddleware为例给大家讲讲中间件在Agent中怎么使用，SummarizationMiddleware是总结摘要中间件（上下文压缩），他的作用是当接近会话次数上限时，自动汇总对话历史记录。

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware,
HumanInTheLoopMiddleware
from langchain_core.messages import HumanMessage
from langchain_core.tools import tool
from langgraph.checkpoint.memory import InMemorySaver

from models import *

# Middleware 中间件
# 添加中间件的方式：在create_agent
'''
agent = create_agent(
    model=qwen_model,
    tools=[],
    middleware=[SummarizationMiddleware(), HumanInTheLoopMiddleware()],
)
'''

# 内置中间件
# LangChain 为常见用例提供预构建的中间件：

# SummarizationMiddleware ：总结摘要的中间件
# 当接近会话次数上限时，自动汇总对话历史记录。
# 非常适合：
# - 持续时间过长的对话超出了上下文窗口。
# - 多轮对话，历史悠久
```

```

# - 在需要保留完整对话上下文的应用中
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware

llm = ChatOpenAI(
    openai_api_base="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=os.getenv("DASHSCOPE_API_KEY"),
    model_name="qwen-max"
)
# 创建短期记忆实例
memory = InMemorySaver()
agent = create_agent(
    model=llm,
    tools=[],
    checkpointer=memory,
    # 中间件列表，可以多个，多个顺序执行
    middleware=[
        SummarizationMiddleware(
            model=llm,
            max_tokens_before_summary=80, # 80个token 会触发 摘要总结
            messages_to_keep=1, # 在总结后保留最后1条消息
            # 可选 summary_prompt=" 可以自定义进行摘要的提示词...",
            summary_prompt="请将以下对话历史进行简洁的摘要，保留关键信息：{messages}"
        ),
    ],
    # 打印Agent执行的过程日志
    debug=True
)

# 单一条件:当tokens> = 4000且消息> = 10时触发
# agent = create_agent(
#     model=llm,
#     tools=[weather_tool, add_tool],
#     middleware=[
#         SummarizationMiddleware(
#             model=llm_other,
#             trigger={"tokens": 4000, "messages": 10},
#             keep={"messages": 20},
#         ),
#     ],
# )
#
# # 多重条件-（任意条件必须满足 - 逻辑“或”）。
# agent2 = create_agent(
#     model="gpt-4o",
#     tools=[weather_tool, add_tool],
#     middleware=[
#         SummarizationMiddleware(
#             model="gpt-4o-mini",
#             trigger=[
#                 {"tokens": 5000, "messages": 3},
#                 {"tokens": 3000, "messages": 6},
#             ],
#             keep={"messages": 20},
#         ),
#     ],
# )

```

```

#     ],
# )

# 模拟长对话触发摘要
print("\n模拟长对话场景...")
demo_messages = [
    "用户询问你是谁",
    "用户计算商品价格：数量10，单价25.5",
    "用户再次询问你能做什么？",
    "用户想要生成一个介绍湖南的文案，要求100字左右，包含三湘四水，人文历史",
    "用户继续询问更多GPU产品信息",
    "用户要求计算2*20"
]

for i, message in enumerate(demo_messages, 1):
    print(f"\n🗨 第{i}轮对话: {message}")
    # 循环调用Agent，模拟多轮对话
    result = agent.invoke({
        "messages": [HumanMessage(content=message)]},
        config={"configurable": {"thread_id": "testsummarizationMiddleware"}}
    )
    # print("执行结果: "+result)

```

3.自定义中间件

自定义中间件Middleware：通过在代理执行流程的特定点运行钩子来构建自定义中间件。

- ✓ 基于装饰器的：适用于单钩中间件的快速简便
- ✓ 基于类的：对于具有多个钩子的复杂中间件来说功能更强大

```

# Node-style: logging before model calls
@before_model
def log_before_model(state: AgentState, runtime: Runtime)
    print(f>About to call model with {len(state['messages'])} messages")
    return None

```

```

class LoggingMiddleware(AgentMiddleware):
    def before_model(self, state: AgentState, runtime: Runtime)
        print(f>About to call model with {len(state['messages'])} messages")
        return None

    def after_model(self, state: AgentState, runtime: Runtime)
        print(f"Model returned: {state['messages'][-1].content}")
        return None

```

具体详细案例参考课程中的代码